Practica 5.

Copia y pega cada prompt tal cual (uno por pregunta). Después de cada respuesta, si algo no queda claro, haz 1–2 preguntas de seguimiento (y si puedes, captura también esa parte en pantalla).

1. PROMPT 1 — Tipología de funciones en Python:"Actúa como tutor de Python para Ciencia de Datos. Explícame detalladamente en qué se diferencian los distintos tipos de funciones en Python (por ejemplo: built-in, definidas por el usuario con def, lambda/anónimas, métodos (instancia/clase/estático), generadoras con yield y funciones async). Para cada tipo: (1) definición breve, (2) un ejemplo corto en Python, y (3) un caso real de cuándo usarla."
2. PROMPT 2 — Arquitectura y diseño de funciones:"Actúa como ingeniero/a de software. ¿Bajo qué criterios técnicos es preferible que una función definida por el usuario reciba parámetros o no? Incluye aspectos como acoplamiento, pureza, dependencias, configuración, testabilidad y reutilización. Además, ¿en qué escenarios es mejor retornar un valor frente a solo ejecutar una acción con efectos secundarios? Da ejemplos concretos en Python."
3. PROMPT 3 — UDF vs. funciones lambda:"Compara funciones definidas por el usuario (UDF con def) vs funciones lambda en Python. ¿En qué situaciones específicas es preferible usar def en lugar de lambda? Explica el impacto en legibilidad, depuración, documentación, pruebas y reutilización. Incluye un ejemplo con lambda y su refactor a def (y explica por qué mejora)."

Prompt 1.

1. Funciones Built-in (Integradas)

- **Definición breve:** Son funciones que ya vienen predefinidas en Python y están siempre disponibles para usar sin necesidad de importar nada. Son la base de muchas operaciones.
- **Ejemplo corto:**

```
print(len("ciencia de datos"))
print
```

2 lines (2 loc) · 63 B

- **Caso real de uso:** Las usarás constantemente para tareas básicas como imprimir resultados (`print`), calcular la longitud de una lista o string (`len`), convertir tipos de datos (`int`, `str`, `float`), sumar elementos (`sum`), encontrar el máximo/mínimo (`max/min`) en tus datasets, o incluso para depurar tus programas.

2. Funciones Definidas por el Usuario (`def`)

- **Definición breve:** Son las funciones "normales" que tú mismo creas para encapsular un bloque de código reutilizable. Te permiten organizar tu lógica y evitar la repetición.
- **Ejemplo corto:**

```
def calcular_media(lista_numeros):
    """Calcula la media de una lista de números."""
    if not lista_numeros:
        return 0
    return sum(lista_numeros) / len(lista_numeros)
datos = [15, 22, 18, 25, 30]
print(calcular_media(datos))
```

8 lines (7 loc) · 268 B

- **Caso real de uso:** ¡Son el pan de cada día en Ciencia de Datos! Las usarás para:
  - **Limpieza y preprocesamiento de datos:** Crear funciones para manejar valores nulos, estandarizar formatos, aplicar transformaciones complejas a columnas específicas de un DataFrame.
  - **Ingeniería de características (Feature Engineering):** Generar nuevas características a partir de las existentes.
  - **Crear modelos personalizados:** Implementar funciones para la lógica de un algoritmo de ML o una métrica de evaluación específica.
  - **Visualización:** Preparar datos antes de graficarlos.

3. Funciones Lambda (Anónimas)

- **Definición breve:** Son funciones pequeñas y anónimas (no tienen nombre) que se definen en una sola línea. Son ideales para operaciones sencillas que no necesitas reutilizar en muchos lugares.
- **Ejemplo corto:**

```
cuadrado = lambda x: x * x
print(cuadrado(7))
# Uso común con funciones de orden superior
numeros = [1, 2, 3, 4, 5]
pares = list(filter(lambda x: x % 2 == 0, numeros))
print(pares)
```

7 lines (6 loc) · 206 B

- **Caso real de uso:** Las verás muchísimo en Ciencia de Datos, especialmente con librerías como Pandas:
  - **Aplicar transformaciones rápidas a DataFrames:** Con `df.apply()` o `df.map()` para operaciones simples en una columna sin definir una función `def` separada. Por ejemplo, `df['columna'] = df['columna'].apply(lambda x: x * 10)`.
  - **Filtrar o ordenar datos:** Usar lambdas como clave en `sort()` o `sorted()`, o con `filter()`.

4. Métodos (de Instancia, de Clase, Estáticos)

Cuando trabajas con Programación Orientada a Objetos (POO), las funciones dentro de una clase se llaman métodos.

a. Métodos de Instancia

- **Definición breve:** Operan sobre una instancia específica de la clase y tienen acceso a los atributos de esa instancia (a través de `self`). Son el tipo de método más común.
- **Ejemplo corto:**

```
class AnalizadorEstadistico:
    def __init__(self, datos):
        self.datos = datos
    def get_media(self):
        return sum(self.datos) / len(self.datos) if self.datos else 0
dataset_1 = AnalizadorEstadistico([10, 20, 30])
print(dataset_1.get_media
```

9 lines (7 loc) · 288 B

- **Caso real de uso:**
  - **Clases para modelos de ML:** Un método `fit()` que entrena el modelo o un método `predict()` que hace predicciones, utilizando los datos y parámetros específicos de esa instancia del modelo.
  - **Objetos de datos complejos:** Si creas una clase para representar un experimento o un cliente, los métodos de instancia operarían sobre las propiedades específicas de ese experimento/cliente.

b. Métodos de Clase (`@classmethod`)

- **Definición breve:** Operan sobre la clase misma, no sobre una instancia particular. Reciben la clase como primer argumento (por convención, `cls`). Se usan a menudo para crear constructores alternativos.
- **Ejemplo corto:**

```
class ProcesadorCSV:
    def __init__(self, ruta_archivo):
        self.ruta_archivo = ruta_archivo
    @classmethod
    def desde_url(cls, url):
        # Lógica para descargar y guardar CSV de una URL
        print(f"Descargando CSV de: {url}")
        return cls(f"temp_data_from_{url.split('/')[-1]}") # Crea una instancia
csv_url = "http://example.com/data.csv"
proc_url = ProcesadorCSV.desde_url(csv_url)
print(proc_url.ruta_archivo
```

- **Caso real de uso:**
  - **Cargar datos de diferentes fuentes:** Podrías tener un método de clase `from_csv(cls, path)` y otro `from_database(cls, connection_string)` que crean instancias de una clase `Dataset` a partir de distintas fuentes de datos.
  - **Configuración global:** Métodos que operan sobre la configuración o propiedades generales de una clase, no ligadas a una instancia.

c. Métodos Estáticos (`@staticmethod`)

- **Definición breve:** Son funciones dentro de una clase que no operan ni sobre la instancia ni sobre la clase. Son esencialmente funciones normales que, por razones lógicas u organizativas, tienen sentido que vivan dentro del ámbito de esa clase. No reciben `self` ni `cls`.
- **Ejemplo corto:**

```
class UtilidadesMatematicas:
    @staticmethod
    def es_par(numero):
        return numero % 2 == 0
print(UtilidadesMatematicas.es_par(4))
print(UtilidadesMat
```

7 lines (6 loc) · 204 B

- **Caso real de uso:**
  - **Funciones auxiliares:** Métodos para tareas genéricas que están relacionadas con la clase, pero no necesitan acceder a sus atributos. Por ejemplo, en una clase `ModeloML`, un método estático `normalizar_vector(vector)` que normaliza un array de números, ya que la normalización es una operación matemática general que no depende de los datos o estado específico del modelo.
  - **Validaciones:** Validar un formato de entrada sin necesidad de crear una instancia de la clase.

5. Funciones Generadoras (`yield`)

- **Definición breve:** Son un tipo especial de función que, en lugar de devolver un valor y terminar, "pausa" su ejecución y devuelve un valor usando la palabra clave `yield`. Cuando se le pide el siguiente valor, la función reanuda su ejecución desde donde la dejó. Esto las hace muy eficientes para trabajar con secuencias de datos grandes, ya que no cargan todo en memoria a la vez.
- **Ejemplo corto:**

- **Ejemplo corto:**

```
def generar_cuadrados(n):
    for i in range(n):
        yield i * i
# Esto no ejecuta la función, sino que crea un objeto generador
mis_cuadrados = generar_cuadrados(5)
print(next(mis_cuadrados)) # 0
print(next(mis_cuadrados)) # 1
# También puedes iterar sobre ellos
for cuadrado in generar_cuadrados(3):
    print(cu
```

13 lines (10 loc) · 380 B

- **Caso real de uso:**
  - **Procesar datasets muy grandes:** Si tienes un archivo CSV o un conjunto de imágenes tan grande que no cabe en la memoria RAM, puedes usar un generador para leerlo línea por línea o en "batches" pequeños, procesando y liberando memoria a medida que avanzas. Esto es clave en Deep Learning para cargar datos de entrenamiento.
  - **Simulaciones:** Generar secuencias infinitas o muy largas de datos simulados sin ocupar toda la memoria.
  - **Stream de datos:** Trabajar con datos que llegan de forma continua (streaming) de una API.

6. Funciones Asíncronas (`async/await`)

- **Definición breve:** Son funciones diseñadas para realizar operaciones de E/S (entrada/salida, como leer un archivo, acceder a una base de datos o hacer una solicitud web) de manera concurrente sin bloquear el hilo principal de ejecución. Usan `async` para definirlas y `await` para esperar de forma no bloqueante la finalización de una operación asíncrona.
- **Ejemplo corto:**

```
import asyncio
async def descargar_dato_remoto(url):
    print(f"Iniciando descarga de {url}...")
    await asyncio.sleep(2)  # Simula una operación de red que toma tiempo
    print(f"Descarga de {url} completada.")
    return f"Datos de {url}"
async def main():
    # Ejecuta varias descargas "simultáneamente"
    resultado_1 = await descargar_dato_remoto("api_datos_1")
    resultado_2 = await descargar_dato_rem  "api_datos_2")
    print(f"Resultados obtenidos: {resultado_1}, {resultado_2}")
```

6. Funciones Asíncronas (`async`/`await`)

- **Definición breve:** Son funciones diseñadas para realizar operaciones de E/S (entrada/salida, como leer un archivo, acceder a una base de datos o hacer una solicitud web) de manera concurrente sin bloquear el hilo principal de ejecución. Usan `async` para definirlas y `await` para esperar de forma no bloqueante la finalización de una operación asíncrona.
- **Ejemplo corto:**

```
import asyncio
async def descargar_dato_remoto(url):
    print(f"Iniciando descarga de {url}...")
    await asyncio.sleep(2)  # Simula una operación de red que toma tiempo
    print(f"Descarga de {url} completada.")
    return f"Datos de {url}"
async def main():
    # Ejecuta varias descargas "simultáneamente"
    resultado_1 = await descargar_dato_remoto("api_datos_1")
    resultado_2 = await descargar_dato_remoto("api_datos_2")
    print(f"Resultados obtenidos: {resultado_1}, {resultado_2}")
# asyncio.run(main()) #
```

15 lines (12 loc) · 601 B

- **Caso real de uso:**
  - **Web Scraping y APIs:** Cuando necesitas hacer cientos o miles de solicitudes HTTP a diferentes URLs o APIs para recolectar datos. Las funciones asíncronas te permiten hacer esto mucho más rápido que si esperaras a que cada solicitud terminara secuencialmente.
  - **Procesamiento de datos en tiempo real:** Para sistemas que necesitan reaccionar rápidamente a eventos o datos entrantes, como la ingestión de datos de sensores o feeds de redes sociales.
  - **Acceso a bases de datos:** Si tu aplicación de Ciencia de Datos necesita interactuar con una base de datos grande, las operaciones asíncronas pueden mejorar drásticamente la capacidad de respuesta.

Espero que esta explicación detallada te sea de mucha ayuda en tu camino por la Ciencia de Datos. ¡Saber cuándo usar cada tipo de función te dará un superpoder para escribir código más limpio, eficiente y robusto! ¿Hay algún tipo en particular en el que te gustaría profundizar más?

Prompt 2.

¿Cuándo una función debe recibir parámetros y cuándo no?

La decisión de que una función reciba parámetros o no depende de varios factores clave que afectan la calidad del código:

- **Acoplamiento (Coupling):**
  - **Con parámetros:** Reduce el acoplamiento. La función es más independiente y no necesita conocer el estado global o de otras partes del sistema para operar. Toda la información que necesita le es explícitamente pasada.
    - **Ejemplo:** Una función que calcula el área de un círculo. Necesita el radio.
  - **Sin parámetros:** Aumenta el acoplamiento. La función debe acceder a variables globales, constantes predefinidas o estados de objetos externos. Esto la hace más difícil de entender y modificar de forma aislada.
    - **Ejemplo:** Una función que imprime un mensaje de bienvenida que accede directamente a una variable global `NOMBRE_USUARIO`.
- **Pureza (Purity):**
  - **Con parámetros (funciones puras):** Si una función con parámetros no tiene efectos secundarios (no modifica estados externos, no hace I/O, etc.) y siempre devuelve el mismo resultado para las mismas entradas, es una función pura. Son el "Santo Grial" del desarrollo funcional: fáciles de razonar, probar y optimizar.
    - **Ejemplo:** `def sumar(a, b): return a + b`
  - **Sin parámetros (funciones impuras):** Las funciones que no toman parámetros (o que los toman pero aun así acceden a estados externos) suelen ser impuras porque sus resultados pueden depender del estado externo del sistema en un momento dado.
    - **Ejemplo:** `def obtener_hora_actual(): return datetime.now()` (depende del tiempo del sistema).
- **Dependencias:**
  - **Con parámetros:** Las dependencias de la función son explícitas en su firma. Es fácil ver qué necesita para funcionar.
  - **Sin parámetros:** Las dependencias son implícitas, lo que puede llevar a "acciones mágicas" donde la función parece obtener datos de la nada, dificultando la depuración y el mantenimiento.
- **Configuración:**
  - **Con parámetros:** Permite una configuración flexible en tiempo de ejecución. Puedes llamar a la misma función con diferentes valores para obtener comportamientos variados.
  - **Sin parámetros:** La configuración suele estar fijada en el código o requiere la modificación de estados globales antes de la llamada, lo que es os flexible y más propenso a errores.

- **Testabilidad:**
  - **Con parámetros:** Las funciones puras con parámetros son triviales de probar. Solo necesitas proporcionar entradas y verificar las salidas, sin preocuparte por el estado del sistema.
    - **Ejemplo:** `assert sumar(2, 3) == 5`
  - **Sin parámetros:** Probar estas funciones es más complejo, ya que a menudo requieren configurar un entorno específico o "mockear" dependencias externas para asegurar un comportamiento predecible.
- **Reutilización:**
  - **Con parámetros:** Las funciones con parámetros bien definidos son altamente reutilizables en diferentes contextos y proyectos, ya que son autónomas y no están atadas a un entorno específico.
  - **Sin parámetros:** Son menos reutilizables, ya que sus dependencias implícitas pueden no existir en otro contexto o pueden comportarse de manera diferente.

En resumen: Como regla general, es *casi siempre preferible* que una función reciba los parámetros que necesita para realizar su trabajo. Esto conduce a un código más modular, predecible, fácil de probar y reutilizar. Las funciones sin parámetros deben usarse con cautela, principalmente para acciones muy específicas y sin estado interno, o cuando interactúan con un entorno global bien definido (como un singleton o una configuración global explícita).

¿Cuándo retornar un valor vs. solo ejecutar una acción con efectos secundarios?

Esta decisión se basa en si la función tiene un **propósito principal de cálculo/información** o un **propósito principal de acción/modificación**.

Escenario 1: Retornar un valor (función con salida)

Es preferible retornar un valor cuando el propósito principal de la función es:

1. **Calcular o producir un resultado:** La función realiza alguna computación y el valor devuelto es el producto de esa computación. **Ejemplo en Python:**

```
def calcular_impuesto(ingreso_anual: float, tasa: float) -> float:
    """Calcula el impuesto basado en el ingreso y la tasa."""
    return ingreso_anual * tasa
impuesto_a_pagar = calcular_impuesto(50000, 0.15)
print(
```

6 lines (5 loc) · 309 B

2. **Consultar o recuperar información:** La función accede a datos (de una base de datos, API, archivo, etc.) y los devuelve. **Ejemplo en Python:**

```python
def obtener_datos_usuario(user_id: int) -> dict:
    """Simula la obtención de datos de usuario de una DB."""
    # En un escenario real, esto interactuaría con una base de datos.
    if user_id == 1:
        return {"id": 1, "nombre": "Alice", "email": "alice@example.com"}
    return {}
usuario = obtener_datos_usuario(1)
if usuario:
    print(f"Usuario encontrado: {usuario['nombre']}") # Output: Usuario encontrado
```

10 lines (9 loc) · 464 B

3. **Indicar un estado o éxito/fallo de una operación:** A menudo, las funciones que realizan acciones también devuelven un booleano o un código de estado para informar sobre el resultado de la operación. **Ejemplo en Python:**

```python
def validar_email(email: str) -> bool:
    """Verifica si el formato del email es válido."""
    return "@" in email and "." in email
if validar_email("test@example.com"):
    print("Email válido.") # Output: Email válido.
else:
    print("Email inválido.")
```

8 lines (7 loc) · 290 B

Escenario 2: Solo ejecutar una acción con efectos secundarios (función sin retorno explícito)

Es preferible que una función *no retorne explícitamente un valor* (implícitamente retorna `None` en Python) cuando su propósito principal es:

1. **Realizar una acción o modificar el estado de algo:** La función existe para cambiar el mundo exterior (imprimir en consola, guardar en base de datos, modificar un objeto mutable, etc.). **Ejemplo en Python:**

```python
def saludar(nombre: str):
    """Imprime un saludo en la consola."""
    print(f"¡Hola, {nombre}!")
saludar("Mundo") # Output: ¡Hola, Mundo!
```

5 lines (4 loc) · 159 B

2. **Modificar un objeto mutable pasado como argumento:** Si la función modifica directamente un objeto que le fue pasado por referencia, a menudo no necesita retornar nada. **Ejemplo en Python:**

```
def agregar_elemento(lista_mutable: list, elemento: any):
    """Agrega un elemento a una lista existente."""
    lista_mutable.append(elemento)
mi_lista = [1, 2]
agregar_elemento(mi_lista, 3)
print(mi_lista) # Output: [1, 2,
```

7 lines (6 loc) · 253 B

Nota: A veces, para mayor fluidez (chaining), incluso estas funciones pueden retornar `self` en programación orientada a objetos o la lista modificada.

3. **Encadenar efectos secundarios complejos:** Cuando una función coordina múltiples operaciones que ya tienen sus propios efectos secundarios o retornos, la función coordinadora puede no necesitar retornar un valor propio. **Ejemplo en Python:**

```
def procesar_pago(cantidad: float, metodo: str):
    """Simula un procesamiento de pago, que incluye enviar notificaciones."""
    # Lógica para interactuar con una pasarela de pago (efecto secundario)
    print(f"Procesando pago de {cantidad} con {metodo}...")
    # Lógica para enviar una confirmación por email (otro efecto secundario)
    print("Enviando confirmación de pago...")
procesar_pago(100.0, "Tarjeta de Crédito")
# Output:
# Procesando pago de 100.0 con Tarjeta de Crédito...
# Enviando confirmación de
```

11 lines (10 loc) · 573 B

En resumen:

- **Retorna un valor** si la función está diseñada para **producir datos o información**. Esto permite que el llamador use esos datos.
- **No retorna un valor explícitamente** si la función está diseñada para **realizar una acción o un efecto secundario** (modificar estado, hacer I/O). Su impacto es la acción que realiza, no un valor que devuelve.

Mantener esta distinción ayuda a crear un código más claro, modular y fácil de entender.

Prompt 3.

¿Cuándo es preferible usar `def` en lugar de `lambda`?

En general, `def` es preferible cuando necesitas funciones que sean más complejas, requieran múltiples expresiones o sentencias, o cuando la función debe ser fácilmente legible, depurable, documentada, testeable y reutilizable.

Aquí te detallo el impacto en cada área:

- **Legibilidad:** Las funciones `def` son mucho más legibles porque permiten múltiples líneas de código, nombres descriptivos, y comentarios. Las `lambda` están diseñadas para ser concisas y de una sola línea, lo que las hace menos legibles si la lógica es un poco intrincada.
- **Depuración:** Depurar una función `def` es sencillo. Puedes poner puntos de interrupción, inspeccionar variables y seguir el flujo de ejecución. Las `lambda` son anónimas y no permiten sentencias, lo que las hace casi imposibles de depurar en el sentido tradicional. Si algo falla en una `lambda`, puede ser muy difícil rastrear el origen del error.
- **Documentación:** Las funciones `def` pueden tener "docstrings" que explican su propósito, argumentos y lo que retornan. Esto es crucial para el mantenimiento del código. Las `lambda` no pueden tener docstrings.
- **Pruebas:** Probar una función `def` es una práctica estándar. Puedes importarla y escribir pruebas unitarias para asegurar su correcto funcionamiento. Las `lambda`, al ser anónimas y a menudo definidas en línea, son mucho más difíciles de probar de forma aislada.
- **Reutilización:** Las funciones `def` tienen un nombre y existen en el ámbito donde se definen, lo que facilita su llamada y reutilización en diferentes partes del código. Las `lambda` suelen ser funciones de un solo uso o que se pasan como argumentos a otras funciones, y no están destinadas a ser reutilizadas ampliamente por su nombre.

Ejemplo: `lambda` y su refactor a `def`

Imagina que tienes una lista de diccionarios que representan usuarios y quieres ordenarlos por nombre de usuario, pero con una lógica especial: si el nombre de usuario es "admin", siempre debe ir al final.

Versión con `lambda` (menos ideal):

```
usuarios = [
    {"username": "alice", "id": 1},
    {"username": "admin", "id": 99},
    {"username": "bob", "id": 2},
    {"username": "charlie", "id": 3},
]
# Ordenar los usuarios, poniendo "admin" al final
usuarios_ordenados = sorted(usuarios, key=lambda u: (1 if u["username"] == "admin" else 0, u["
print(usuarios_ordenados)
```

11 lines (9 loc) · 344 B

Salida:

```
[{'username': 'alice', 'id': 1}, {'username':
```

1 lines (1 loc) · 129 B

Aquí, la `lambda` funciona, pero la lógica dentro de ella (`(1 if u["username"] == "admin" else 0, u["username"])`) ya empieza a ser un poco densa para una sola línea.

Refactor a `def` (preferible):

```
    {"username": "bob", "id": 2},
    {"username": "charlie", "id": 3},
]
def obtener_clave_ordenacion_usuario(usuario):
    """
    Genera una clave de ordenación para un usuario.
    Los usuarios con username 'admin' siempre van al final.
    """
    if usuario["username"] == "admin":
        return (1, usuario["username"]) # El 1 asegura que 'admin' vaya después de todo lo qu
    else:
        return (0, usuario["username"]) # El 0 asegura que los demás vayan antes, ordenados a
# Ordenar los usuarios usando la función 'def'
usuarios_ordenados_refactor = sorted(usuarios, key=obtener_clave_ordenacion_usuario)
print(usuarios_orden
```

21 lines (18 loc) · 769 B

Salida:

```
[{'username
```

1 lines (1 loc) · 129 B

¿Por qué la versión `def` mejora?

1. **Legibilidad:** El nombre `obtener_clave_ordenacion_usuario` es autoexplicativo. El código dentro de la función está bien estructurado con un `if/else`, lo que facilita entender la lógica del ordenamiento.
2. **Depuración:** Si hubiera un error en la lógica de ordenamiento, sería trivial depurar `obtener_clave_ordenacion_usuario`.
3. **Documentación:** Tiene un `docstring` que explica claramente lo que hace la función y cómo genera la clave de ordenación. Esto es invaluable para cualquiera que lea o mantenga el código.
4. **Pruebas:** Puedes escribir pruebas unitarias para `obtener_clave_ordenacion_usuario` de forma aislada, asegurando que siempre devuelva la clave correcta para diferentes tipos de usuarios.
5. **Reutilización:** Aunque en este ejemplo solo se usa una vez, si en el futuro necesitaras aplicar esta misma lógica de ordenamiento en otro lugar, simplemente llamarías a `obtener_clave_ordenacion_usuario`.

En resumen, usa `lambda` para funciones pequeñas y anónimas que hacen algo muy simple, generalmente cuando la pasas como argumento a otra función (como `map`, `filter`, `sorted`, `functools.reduce`). Para cualquier otra cosa, especialmente si la lógica es un poco más compleja o si la función es un componente importante de tu programa, `def` es el camino a seguir.